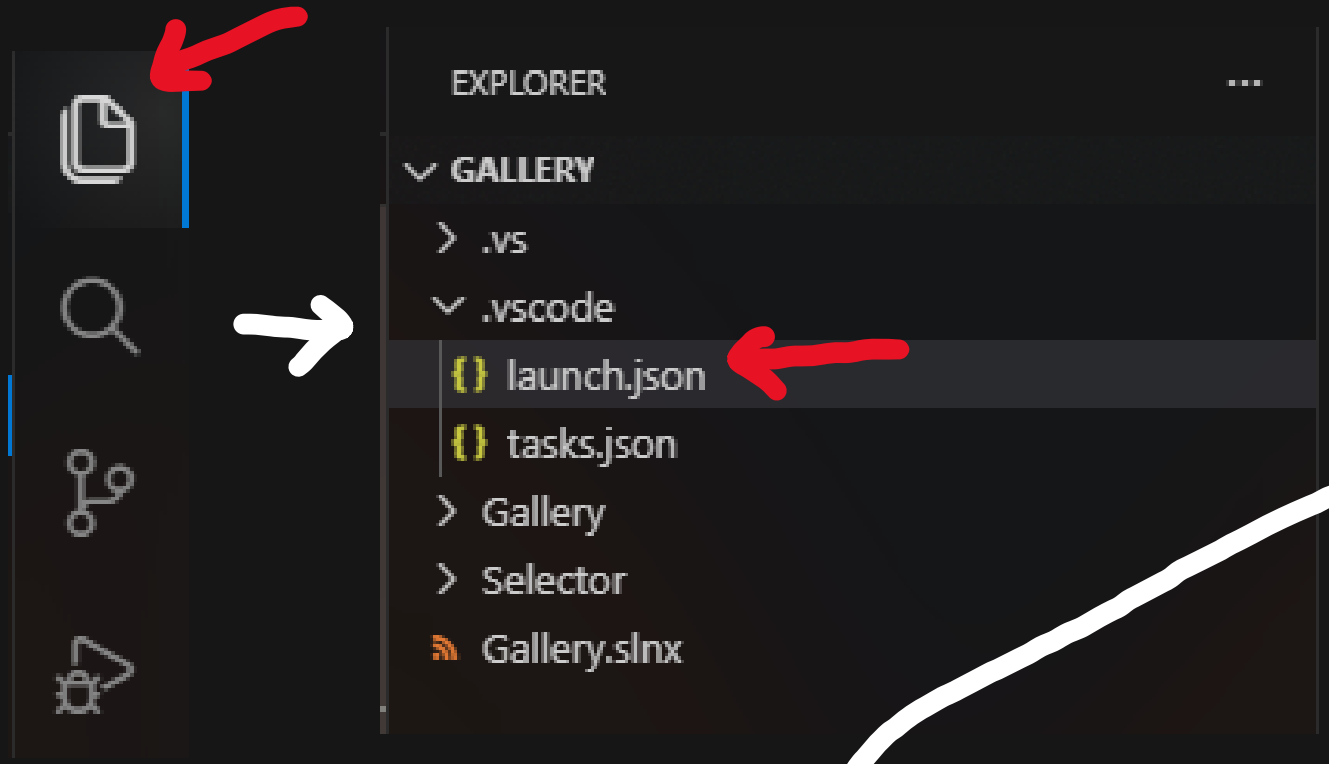
Console Formatting

By Ignacy

External Terminal




```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": ".NET Core Launch (console)",
            "type": "coreclr",
            "request": "launch",
            "preLaunchTask": "build",
            "program": "${workspaceFolder}/...",
            "args": [],
            "cwd": "${workspaceFolder}",
            "console": "internalConsole",
            "stopAtEntry": false
        },
        {
            "name": ".NET Core Attach",
            "type": "coreclr",
            "request": "attach"
        }
    ]
}
```

`"console": "internalConsole"` → `"console": "externalTerminal"`

External Terminal

```
static void Main(string[] args)
{
    // Stuff...

    // Pause program until any key
    Console.ReadKey(intercept: true);
}
```

Two ways of color

```
Console.WriteLine("Hello, World!");
Console.ForegroundColor = ConsoleColor.Red;
Console.WriteLine("Hello, World! (but in red)");
Console.ResetColor();
Console.WriteLine("Hello, World! (but back in default colors)");
```

```
Console.WriteLine("Hello, World!");
Console.WriteLine("\e[31mHello, World! (but in red)"); // '\e[31m' sets foreground to red
Console.WriteLine("\e[39mHello, World! (but back in default colors)"); // '\e[39m' resets foreground
```

Both output:

```
Hello, World!
Hello, World! (but in red)
Hello, World! (but back in default colors)
```

ANSI escape codes

```
// '\e[31m': ANSI escape code for text with a red foreground
Console.WriteLine("\e[31mHello, World!");   ➔   Hello, World!

// '\e[0m': escape code to reset all text formatting
Console.Write("\e[0m");

// '\e[41m': code for text with a red background
Console.WriteLine("\e[41mHello, World! 2x");   ➔   Hello, World! 2x

Console.Write("\e[0m");

// '\e[3m': italicized text
Console.WriteLine("\e[3mHello, World! 3x");   ➔   Hello, World! 3x

Console.Write("\e[0m");

// '\e[1m': bold text
// '\e[4m': underlined text
Console.WriteLine("\e[1m\e[4mHello, World! 4x");   ➔   Hello, World! 4x
```

ANSI escape code format

CSI

(\e or \x1b) + [

Argument(s)

0 ➔ reset formatting
1 ➔ bold text
3 ➔ italicize text
etc.

Command

m ➔ text formatting
t ➔ window management
H ➔ cursor position
etc.

```
"\e[<argument(s)><command>"

<command>(<argument(s)>);
```

```
"\e[0m"

FormatText(0);
```

Why use ANSI over Console properties

- More concise

```
Console.ForegroundColor = ConsoleColor.Red;
Console.WriteLine("Red");
Console.ResetColor();
```

↓ becomes just

```
Console.WriteLine("\e[31mRed\e[0m");
```

- More functionality

```
Console.Write("\e[3m"); // Make text italicized

// Console has no equivalent functionality – no Console.Italicize() or anything
```

- Can be harder to read, though

Colors with ANSI

Predefined colors		
Foreground Argument (starts with 3)	Background Argument (starts with 4)	Color
30	40	Black
31	41	Red
32	42	Green
33	43	Yellow
34	44	Blue
35	45	Magenta
36	46	Cyan
37	47	White
39	49	Reset color

`"\e[<argument>m"`

38 - Foreground
48 - Background

8-bit

`"\e[<38/48>;5;<index>m"`

24-bit

`"\e[<38/48>;2;<R>;<G>;<B>m"`

Questions